

강의 2 실험

xv6의 C와 메모리 · MIT 6.1810 강의 2를 xv6에서 직접 돌려 보며

MIT 6.1810(2026 가을)의 두 번째 강의는 C 입문이 아니라 **xv6에서 C가 메모리를 어떻게 쓰는가**를 다룬다. 메모리 배치, 포인터, 배열, 문자열, 연결 리스트, 비트 연산 순서로 진행하고, 숙제는 xv6 교재가 아니라 K&R 『The C Programming Language』의 포인터 장이다. 이 문서는 강의의 예제(ptr.c, array.c, str.c)와 직접 만든 probe.c를 xv6에서 돌리고, 컴파일 결과물과 실행 중인 커널을 objdump와 gdb로 들여다본 기록이다. 찍힌 주소는 모두 실제로 나온 값이다.

2026년 10월 7일 기록. 대상: xv6-riscv 06aad25 + 강의 예제(lec1 브랜치)의 작업용 복사본, 교재 2026-09-23판. "tmux 캡처"는 xv6, gdb, 호스트 셸을 tmux 창에서 실제로 돌린 뒤 **tmux capture-pane**으로 받은 글자를 창 모양으로 그린 것이다. 글자는 고치지 않았다. 밑줄 친 쪽 번호를 누르면 교재 PDF의 그 쪽이 열린다.

1. 범위

강의 2에는 짝이 되는 교재 장이 없다. 강의 노트(l-c.txt)의 주제를 예제와 xv6 코드, 숙제로 읽는 K&R 절에 대응시키면 다음과 같다.

노트의 주제	실험	함께 볼 xv6 코드	K&R
실행 파일 만들기	objdump, nm	Makefile, user/user.ld, user/ulib.c:start	-
메모리 배치	probe	kernel/exec.c, user/umalloc.c	-
포인터	ptr	-	5.1, 5.2, 6.4
배열, 주소 산술	array	-	5.3, 5.4
문자열	str, gdb	user/ulib.c:strlen, strcmp	5.5
argv, 호출 규약	gdb	kernel/exec.c:101--124	5.6
단일 연결 리스트	gdb	kernel/kalloc.c	6.4
이중 연결 리스트	gdb	kernel/bio.c, kernel/buf.h	6.4
비트 연산	gdb	kernel/fcntl.h, kernel/sysfile.c, kernel/riscv.h	2.9

노트가 교재를 가리키는 곳은 사용자 주소 공간 그림 하나다. 교재 3.6절의 그림 3.4 (39쪽)이다. 숙제에는 32비트 시스템에서 구조체 멤버의 주소를 계산하는 문제가 있는데, 이 문서는 답을 쓰지 않는다. 노트가 언급하는 시연 파일 hw.c는 강의 사이트에서 받을 수 없었다(404).

2. 실습 환경

~/xv6-riscv를 직접 고치지 않으려고 lec1 브랜치의 작업 트리를 work/xv6로 복사했다. 여기에 강의 예제 세 개와 probe.c를 user/에 넣고 Makefile의 UPROGS에 \$U/_ptr, \$U/_array, \$U/_str, \$U/_probe를 더했다. gdb 실험은 QEMU를 -S -gdb tcp::26300으로 띄우고 다른 tmux 창의 riscv64-unknown-elf-gdb로 붙였다(capture-gdb.sh). CPU는 1개다.

사용자 프로그램을 gdb로 멈출 때는 break 대신 hbreak(하드웨어 중단점)를 썼다. gdb로 끼어든 순간 CPU는 커널 페이지 테이블을 쓰고 있어서, 사용자 주소에 중단점 명령어를 써 넣을 수 없기 때문이다.

3. C 코드가 실행 파일이 되기까지

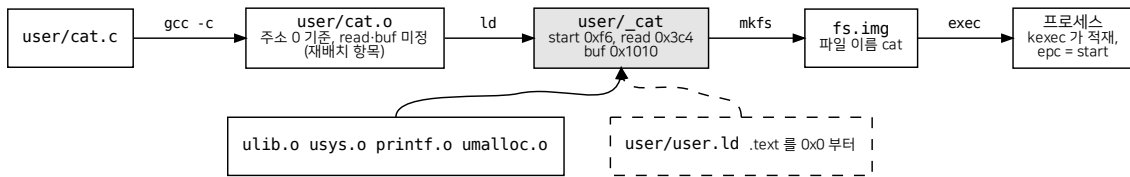


그림 1. cat.c가 프로세스가 되기까지. .o에서는 read와 buf의 주소가 비어 있고, 링크가 끝난 _cat에서 정해진다.

Makefile은 cat.c를 gcc -c로 cat.o로 만들고, xv6의 작은 C 라이브러리 (ULIB = ulib.o usys.o printf.o umalloc.o, Makefile:104)와 함께 ld -T user/user.ld로 링크해 _cat을 만든다 (Makefile:106--107).

```
host shell - Ubuntu

$ od -h user/cat.o | grep -A1 -E "\.(text|data|bss|rodata)"
0 .text          0000012e 0000000000000000 0000000000000000 00000040 2**1
CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
1 .data          00000000 0000000000000000 0000000000000000 0000016e 2**0
CONTENTS, ALLOC, LOAD, DATA
2 .bss           00000200 0000000000000000 0000000000000000 00000170 2**3
ALLOC
3 .rodata.str1.8 00000045 0000000000000000 0000000000000000 00000170 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA

$ od -r user/cat.o | grep -E "CALL|PCREL_HI"
0000000000000018 R_RISCV_PCREL_HI20 .LANCHOR0
0000000000000028 R_RISCV_CALL_PLT read
000000000000003c R_RISCV_CALL_PLT write
0000000000000048 R_RISCV_PCREL_HI20 .LC0
0000000000000052 R_RISCV_CALL_PLT fprintf
000000000000005c R_RISCV_CALL_PLT exit
000000000000007a R_RISCV_PCREL_HI20 .LC1
0000000000000084 R_RISCV_CALL_PLT fprintf
000000000000008e R_RISCV_CALL_PLT exit
00000000000000c4 R_RISCV_CALL_PLT open
00000000000000d2 R_RISCV_CALL_PLT cat
00000000000000dc R_RISCV_CALL_PLT close
00000000000000ec R_RISCV_CALL_PLT exit
00000000000000fc R_RISCV_CALL_PLT cat
0000000000000106 R_RISCV_CALL_PLT exit
0000000000000112 R_RISCV_PCREL_HI20 .LC2
000000000000011c R_RISCV_CALL_PLT fprintf
0000000000000126 R_RISCV_CALL_PLT exit
$
```

tmux 캡처(호스트). cat.o는 모든 섹션의 주소가 0이고, read, write, buf(.LANCHOR0)를 부르는 자리에 재배치 항목이 남아 있다.

od와 nm은 riscv64-unknown-elf-objdump, -nm의 별칭이다. 0x28의 R_RISCV_CALL_PLT read는 “여기에 read의 주소를 채워 넣으라”는 표시다. 링크가 끝난 _cat에서는 그 자리가 채워져 있다.

```

host shell - Ubuntu

$ od -f user/_cat | grep start
start address 0x00000000000000f6
$ od -h user/_cat | grep -A1 -E " \.(text|rodata|data|bss) "
 0 .text      000009b8 0000000000000000 0000000000000000 00001000 2**1
               CONTENTS, ALLOC, LOAD, READONLY, CODE
 1 .rodata    00000069 00000000000000b8 00000000000000b8 000019b8 2**3
               CONTENTS, ALLOC, LOAD, READONLY, DATA
 2 .data      00000000 0000000000001000 0000000000001000 00002000 2**0
               CONTENTS, ALLOC, LOAD, DATA
 3 .bss       00000220 0000000000001000 0000000000001000 00002000 2**3
               ALLOC

$ nm -n user/_cat | grep -wE "cat|main|start|exit|read|buf"
0000000000000000 T cat
000000000000007e T main
00000000000000f6 T start
00000000000003ac T exit
00000000000003c4 T read
0000000000001010 B buf
$

```

tmux 캡처(호스트). _cat의 진입점은 0xf6(start), .text는 0부터, .bss의 buf는 0x1010이다.

```

# user/cat.asm (objdump -S of _cat)
while ((n = read(fd, buf, sizeof(buf))) > 0) {
18: 00001917 auipc s2,0x1
1c: ff890913 addi s2,s2,-8 # 1010 <buf>
...
28: 39c000ef jal 3c4 <read>

```

cat.o의 0x18, 0x28에 남아 있던 두 재배치가 buf=0x1010, read=0x3c4로 채워졌다. user.ld가 .text를 0x0에, .data를 그다음 4KB 경계(0x1000)에 놓으라고 정하기 때문이다. user.ld에 ENTRY가 없어 GNU ld의 기본 규칙대로 start 심볼이 진입점이 되었다. start(user/ulib.c:12)는 main을 부르고 그 반환값으로 exit을 부른다. 그래서 main이 exit을 부르지 않고 return해도 프로세스가 끝난다. 커널의 kexec은 ELF 헤더의 진입점을 p->trapframe->epc에 넣는다(kernel/exec.c:136).

핵심 .o는 주소가 정해지지 않은 조각이고, 링커가 user.ld에 따라 주소를 정하며 빈자리를 채운다. 사용자 프로그램은 main이 아니라 start에서 시작한다.

4. 사용자 프로그램의 메모리 배치

강의는 메모리를 text, data, stack, heap 네 부분으로 설명한다. 각 부분이 실제로 어느 주소에 놓이는지 보려고 작은 프로그램을 만들었다.

```

int initialized = 7;      // .data
int zeroed;              // .bss
char *lit = "literal";    // pointer in .data, string in .rodata

int
main(int argc, char *argv[])
{
    int local = 1;         // stack
    char *heap = malloc(16); // heap (sbrk)
    printf("main          %p\n", main);
    ...                    // lit, &initialized, &zeroed, heap, sbrk(0), &local, argv
}

```

```
xv6 · qemu-system-riscv64

$ probe hello world
main      0x0000000000000000
lit string 0x00000000000000A40
&initialized 0x0000000000001008
&zeroed   0x0000000000001010
heap      0x00000000000013FF0
sbrk(0)    0x00000000000014000
&local    0x0000000000003F6C
argv       0x0000000000003FB0
argv[0]    0x0000000000003FF0 "probe"
argv[1]    0x0000000000003FE0 "hello"
argv[2]    0x0000000000003FD0 "world"
$
```

tmux 캡처. 인자 두 개를 주고 실행했다.

```
host shell · Ubuntu

$ od -h user/_probe | grep -E " \.(text|rodata|data|bss) "
0 .text      0000099e 0000000000000000 0000000000000000 00001000 2**1
1 .rodata    00000c1 00000000000009a0 00000000000009a0 000019a0 2**3
2 .data      00000010 0000000000001000 0000000000001000 00002000 2**3
3 .bss       00000020 0000000000001010 0000000000001010 00002010 2**3
$ nm -n user/_probe | grep -wE "main|start|lit|initialized|zeroed|end"
0000000000000000 T main
00000000000000dc T start
0000000000001000 D lit
0000000000001008 D initialized
0000000000001010 B zeroed
$
```

tmux 캡처(호스트). 같은 실행 파일의 섹션과 심볼. 프로그램이 찍은 main, lit, initialized, zeroed의 주소와 같다.

가상 주소	영역	probe 가 찍은 값
0x4000 – 0x14000	힙 (malloc → sbrk)	heap 0x13ff0, sbrk(0) 0x14000
0x3000 – 0x4000	스택 (1쪽)	argv[0..2] 0x3ff0 0x3fe0 0x3fd0 argv 0x3fb0, &local 0x3fc
0x2000 – 0x3000	가드 페이지	사용자가 접근할 수 없다
0x1010 – 0x1030	.bss (0으로 채움)	&zeroed 0x1010
0x1000 – 0x1010	.data	lit 0x1000, &initialized 0x1008
0x9a0 – 0xa61	.rodata	"literal" 0xa40
0x0 – 0x99e	.text	main 0x0, start 0xdc

그림 2. probe hello world가 찍은 주소로 그린 사용자 주소 공간. 위가 높은 주소다. 교재 그림 3.4와 같은 모양이고, 맨 위의 trapframe과 trampoline은 생략했다.

text부터 bss까지는 실행 파일 그대로다(nm 값과 일치). 나머지는 kexec이 정한다. kexec은 마지막 세그먼트 끝(0x1030)을 4KB로 올림한 0x2000 위에 두 쪽을 더 잡고 (kernel/exec.c:89--91), 아래쪽 하나를 사용자 접근 불가로 바꿔 가드 페이지로 만든다 (exec.c:95). 위쪽 한 쪽(0x3000-0x4000)이 스택이고, sp는 0x4000에서 시작한다. 인자 문자열이 맨 위(0x3ff0, 0x3fe0, 0x3fd0)에 먼저 놓이고, 그 아래에 argv 배열(0x3fb0), 그 아래에 main의 지역 변수(0x3f6c)가 온다. 인자 없이 실행했을 때 &local은 0x3f9c였다. 인자가 스택을 차지한 만큼 지역 변수가 아래로 밀렸다.

힙은 스택 위, 프로세스 크기 $p \rightarrow sz=0x4000$ 부터다. 16바이트를 malloc했는데 $sbrk(0)$ 이 $0x14000$ 이 됐다. `umalloc.c`의 `morecore`가 한 번에 최소 4096개의 Header(16바이트), 즉 64KB를 요청하기 때문이다(`user/umalloc.c:52--54`). 그리고 malloc은 빈 블록의 뒤쪽을 잘라 준다(`umalloc.c:80`). 그래서 반환값이 블록 끝 바로 아래인 $0x13ff0$ 이다.

핵심 `text·data·bss`의 주소는 링커가, 스택과 가드 페이지는 `exec0`이, 힙은 `sbrk`가 정한다. xv6 사용자 프로그램의 `main`은 주소 0에서 시작한다.

5. 포인터

```
xv6 · qemu-system-riscv64
$ ptr
p 0x0000000000001000 q 0x0000000000003FAC
g 11 l 13
10 10
pp 0x0000000000003FA0 0x0000000000001000 11
main: 0x0000000000000000
$
```

tmux 캡처. `ptr.c`의 출력.

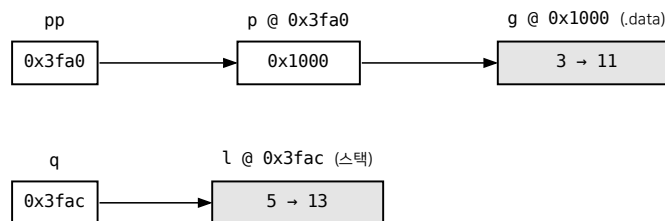


그림 3. `ptr.c`의 포인터. 칸 위는 변수 이름과 그 변수가 놓인 주소, 칸 안은 값이다.

포인터는 주소를 담은 변수다. $p = \&g$ 는 전역 변수 `g`의 주소 $0x1000(.data)$ 을, $q = \&l$ 은 지역 변수 `l`의 주소 $0x3fac$ (스택)를 담는다. $*p = 11$ 로 `g`가 바뀐다. 포인터 변수 `p`도 메모리($0x3fa0$)에 있으므로 $pp = \&p$ 로 그 주소를 다시 담을 수 있고, $**pp$ 는 `g`의 값 11이다. 구조체 포인터는 `ptr->a`로 멤버에 접근하며 $(*ptr).a$ 와 같다. 함수의 주소도 포인터에 담을 수 있다. `main: 0x0`은 앞 절의 배치대로 `main`이 `.text`의 맨 앞에 있기 때문이다.

핵심 포인터 값은 주소일 뿐이고, 타입은 그 주소에서 몇 바이트를 어떻게 읽을지를 정한다.

6. 배열과 문자열

6.1 배열

```
xv6 · qemu-system-riscv64
$ array
1
3 5
a 0x0000000000001010 a1 0x0000000000001014 a2 0x0000000000001018 a2 0x0000000000000001
018
0x0000000000001000 0x0000000000001001
$
```

tmux 캡처. `array.c`의 출력.

$a[1] += 1$ 과 $*(a+2) = 5$ 는 같은 방식의 접근이다. $a+1$ 은 0x1010에서 1이 아니라 4 늘어난 0x1014다. 포인터에 정수를 더하면 가리키는 타입의 크기만큼 움직인다. char 배열 b의 $b+1$ 은 1만 늘었다. 배열은 경계를 검사하지 않고 크기를 바꿀 수도 없다.

6.2 문자열

```

$ str
s 123 s1 123
1 1
3 3
0 0x00000000000003FA8 0x00000000000003FAC
$

```

tmux 캡처. str.c의 출력. 끝의 두 줄은 배열 밖을 읽고 쓰는 부분이다.

C 문자열은 끝에 0('\0')이 붙은 char 배열이다. gdb로 str을 멈춰 두 문자열을 비교했다.

```

(gdb) hbreak str.c:19
Hardware assisted breakpoint 2 at 0x5c: file user/str.c, line 19.
(gdb) c
Continuing.
Breakpoint 2, main (ac=<optimized out>, av=<optimized out>) at user/str.c:19
19     printf("%x %p %p\n", s1[4], s1, &s1[4]);
(gdb) p sizeof(s1)
$1 = 4
(gdb) p sizeof(s)
$2 = 8
(gdb) p s
$3 = 0x978 "123"
(gdb) x/4c s
0x978: 49 '1' 50 '2' 51 '3' 0 '\000'
(gdb) p &s1
$4 = (char (*)[4]) 0x3f88
(gdb) x/8xb s1
0x3f88: 0x31 0x32 0x33 0x00 0x00 0x00 0x00 0x00
(gdb)

```

tmux 캡처(gdb). 19행에서 멈춰 s와 s1을 본 화면.

s1은 4바이트 배열이라 sizeof(s1)이 4, s는 포인터라 8이다. s가 가리키는 "123"은 0x978, 곧 읽기 전용 .rodata에 있다. s1은 스택(0x3f88)에 있고, 네 번째 바이트가 0이다. s1[4]는 배열 밖인 0x3f8c를 읽는다. 이번에는 0이 나왔지만 보장된 값이 아니다. 이어지는 s1[4] = 'D'도 아무 오류 없이 지나갔다. C는 배열 밖 쓰기를 막지 않으므로 다른 변수를 덮어써도 바로 드러나지 않는다. 강의 노트가 혼한 C 버그로 꼽는 buffer overflow가 이것이다.

xv6의 strlen(user/ulib.c:40)은 s[n]처럼 배열 방식으로, strcmp(ulib.c:32)는 *p, p++처럼 포인터 방식으로 0을 만날 때까지 걷는다.

핵심 배열 이름은 첫 원소의 주소로 쓰이지만 sizeof는 배열 전체 크기를 준다. 문자열의 끝은 길이가 아니라 0으로 표시된다.

7. argv와 호출 규약

main(int argc, char *argv[])의 두 인자는 어디서 오는가. str hi로 실행하고 main의 첫 명령어에 하드웨어 중단점을 걸었다.

```

riscv64-unknown-elf-gdb
(gdb) symbol-file user/_str
Reading symbols from user/_str...
(gdb) hbreak *main
Hardware assisted breakpoint 1 at 0x0: file user/str.c, line 8.
(gdb) c
Continuing.
Breakpoint 1, main (ac=2, av=0x3fc0) at user/str.c:8
8      {
(gdb) info registers a0 a1 sp ra
a0          0x2      2
a1          0x3fc0   16320
sp          0x3fb0   0x3fb0
ra          0x8c     0x8c <start+12>
(gdb) info symbol $ra
start + 12 in section .text
(gdb) x/3gx $a1
0x3fc0: 0x00000000000003ff0      0x00000000000003fe0
0x3fd0: 0x0000000000000000
(gdb) x/s *(char **)$a1
0x3ff0: "str"
(gdb) x/s ((char **)$a1)[1]
0x3fe0: "hi"
(gdb)

```

tmux 캡처(gdb).main에 들어온 순간의 레지스터와 argv.

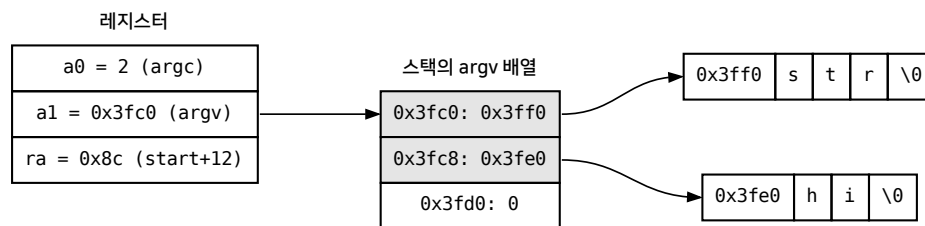


그림 4. str hi의 main이 받은 인자. a1이 스택의 포인터 배열을 가리키고, 배열의 각 칸이 스택 위쪽의 문자열을 가리킨다. 배열 끝은 0이다.

RISC-V 호출 규약에서 함수 인자는 a0부터 a7까지 레지스터로 넘기고, 반환 주소는 ra에 둔다. a0=2는 argc, a1=0x3fc0은 argv다. ra는 start+12로, main이 끝나면 start로 돌아가 exit을 부른다. 이 배열과 문자열은 kexec이 스택에 복사한 것이다. 문자열을 하나씩 위에서부터 놓고 16바이트로 정렬한 뒤 (kernel/exec.c:101--108), 그 주소들의 배열을 아래에 놓고(exec.c:113--117), 배열 주소를 a1에 넣는다(exec.c:124). argc는 exec 시스템 콜의 반환값으로 a0에 들어간다. cat.asm에서 read(fd, buf, 512)를 부르기 직전에도 a0, a1, a2를 채운 뒤 jal한다.

```

22: 8652  mv  a2,s4      # 512
24: 85ca  mv  a1,s2      # buf
26: 854e  mv  a0,s3      # fd
28: ...   jal  3c4 <read>

```

핵심 인자는 a0-a7로 넘어가고, argv는 exec이 스택 맨 위에 만든 포인터의 배열이다.

8. 커널의 연결 리스트

8.1 빈 페이지 목록: 단일 연결 리스트

커널의 물리 페이지 할당기는 빈 4KB 페이지들을 연결 리스트로 관리한다(kernel/kalloc.c). 리스트 노드 struct run은 따로 메모리를 잡지 않고 빈 페이지 자신의 첫 8바이트에 놓인다.

```

struct run { struct run *next; };           // kalloc.c:17
kfree:  r = (struct run *)pa;               // kalloc.c:57
        r->next = kmem.freelist;           // prepend
        kmem.freelist = r;
kalloc: r = kmem.freelist;                  // kalloc.c:74
        if (r) kmem.freelist = r->next;    // take from the front

```

셸이 뜬 뒤 gdb로 끼어들어 리스트를 따라갔다.

```

(gdb) c
Continuing.
^C
Program received signal SIGINT, Interrupt.
scheduler () at kernel/proc.c:441
441      intr_on();
(gdb) p sizeof(struct run)
$1 = 8
(gdb) p kmem.freelist
$2 = (struct run *) 0x87f42000
(gdb) p kmem.freelist->next
$3 = (struct run *) 0x87f54000
(gdb) p kmem.freelist->next->next
$4 = (struct run *) 0x87f53000
(gdb) p (char*)kmem.freelist - (char*)kmem.freelist->next
$5 = -73728
(gdb) set $n = 0
(gdb) set $r = kmem.freelist
(gdb) while $r
>set $n = $n + 1
>set $r = $r->next
>end
(gdb) p $n
$6 = 32545
(gdb) p/x (0x88000000 - (unsigned long)end) / 4096
$7 = 0x7fdf
(gdb)

```

tmux 캡처(gdb). 리스트의 앞 세 노드와 전체 길이.

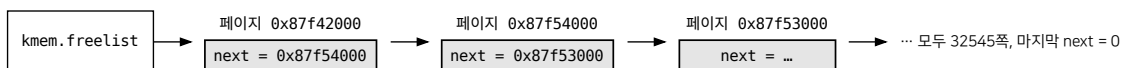


그림 5. 실측한 kmem.freelist의 앞부분. 노드의 주소가 곧 빈 페이지의 주소다.

빈 페이지는 32545쪽이었다. 커널 끝(end)부터 PHYSTOP(0x88000000, kernel/memlayout.h:44)까지는 0x7fdf = 32735쪽이므로, 셸이 뜬 시점까지 190쪽을 쓰고 있었다. 노드 주소가 차례대로가 아닌 것은 부팅 중에 할당과 해제가 섞이면서, 해제된 페이지가 리스트 앞에 다시 붙었기 때문이다.

8.2 버퍼 캐시: 이중 연결 리스트

디스크 블록 캐시(kernel/bio.c)는 struct buf의 prev, next(kernel/buf.h)로 원형 이중 연결 리스트를 만든다. bcache.head.next가 가장 최근에 쓴 버퍼, head.prev가 가장 오래된 버퍼다. brelse는 다 쓴 버퍼를 리스트에서 떼어 맨 앞에 다시 붙인다.

```

if (b->refcnt == 0) {
    b->next->prev = b->prev;           // unlink
    b->prev->next = b->next;
    b->next = bcache.head.next;       // insert after head
    b->prev = &bcache.head;
    bcache.head.next->prev = b;
}

```

```

bcache.head.next = b;
}

```

```

riscv64-unknown-elf-gdb
(gdb) p &bcache.head
$8 = (struct buf *) 0x8001da50 <bcache+33384>
(gdb) p bcache.head.next->blockno
$9 = 47
(gdb) p bcache.head.next->next->blockno
$10 = 410
(gdb) p bcache.head.next->prev == &bcache.head
$11 = 1
(gdb) p bcache.head.prev->next == &bcache.head
$12 = 1
(gdb)

```

tmux 캡처(gdb). 셸이 뜬 직후 리스트의 앞쪽, 양쪽 연결이 서로를 가리킨다.

맨 앞이 아닌 버퍼가 **brelease**되는 순간을 잡으려고 조건부 중단점을 걸고 xv6에서 cat README를 실행했다.

```

riscv64-unknown-elf-gdb
(gdb) b brelease if b->refcnt == 1 && bcache.head.next != b
Breakpoint 1 at 0x80002c52: file kernel/bio.c, line 118.
(gdb) c
Continuing.
Breakpoint 1, brelease (b=b@entry=0x800160b0 <bcache+2248>) at kernel/bio.c:118
118     {
(gdb) p b->blockno
$1 = 33
(gdb) p bcache.head.next->blockno
$2 = 47
(gdb) finish
Run till exit from #0 brelease (b=b@entry=0x800160b0 <bcache+2248>) at kernel/bio.c:118
ilock (ip=ip@entry=0x8001dffb <itable+296>) at kernel/fs.c:315
315     ip->valid = 1;
(gdb) p bcache.head.next->blockno
$3 = 33
(gdb) p bcache.head.next->next->blockno
$4 = 47
(gdb)

```

tmux 캡처(gdb). 블록 33의 버퍼가 **brelease**된 뒤 맨 앞으로 왔고, 원래 맨 앞이던 47이 두 번째가 됐다.

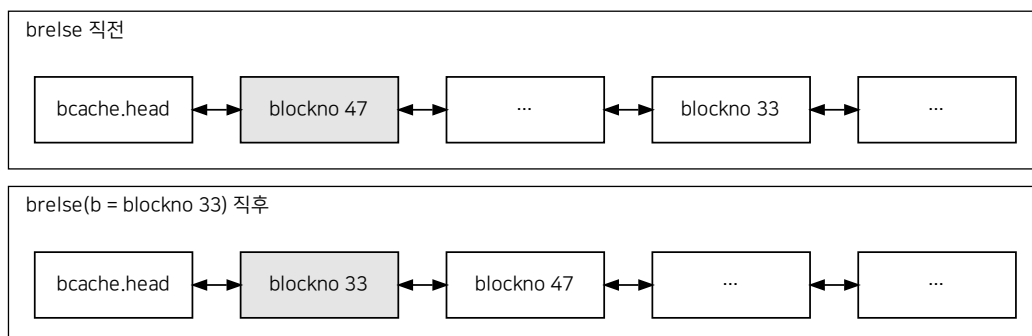


그림 6. 실측한 **brelease** 전후. 양쪽 화살표는 **next**와 **prev** 한 쌍이다.

핵심 **kalloc**은 빈 페이지 안에 **next**를 넣어 단일 리스트를 만들고, 버퍼 캐시는 **prev/next**로 이중 리스트를 만들어 최근에 쓴 것을 맨 앞에 둔다.

9. 비트 연산

9.1 플래그 묶기

open의 모드는 비트 하나씩을 뜻하는 상수를 |로 묶어 넘긴다(kernel/fcntl.h). O_WRONLY=0x001, O_RDWR=0x002, O_CREATE=0x200, O_TRUNC=0x400이다. usertests.c의 open("small", O_CREATE | O_RDWR)은 0x202를 넘긴다. 커널의 sys_open은 &로 비트를 하나씩 확인한다.

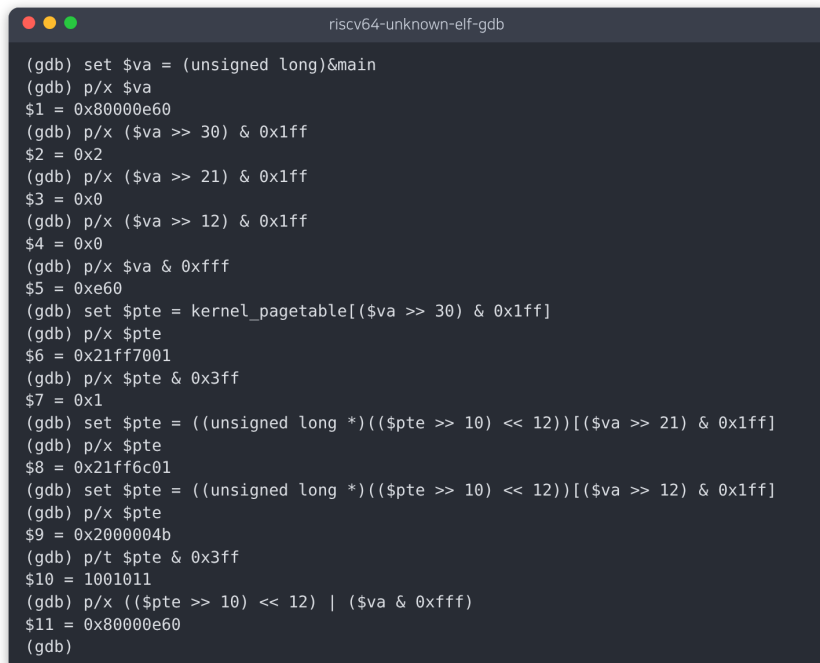
```
if (omode & O_CREATE) { // sysfile.c:343
f->readable = !(omode & O_WRONLY); // sysfile.c:384
f->writable = (omode & O_WRONLY) || (omode & O_RDWR); // sysfile.c:385
if ((omode & O_TRUNC) && ip->type == T_FILE) { // sysfile.c:387
```

9.2 주소 쪼개기

페이지 테이블 코드는 시프트와 마스크로 주소를 쪼갬다(kernel/riscv.h).

```
#define PTE_V (1L << 0) #define PTE_R (1L << 1) #define PTE_X (1L << 3) // :395-398
#define PA2PTE(pa) (((uint64)pa) >> 12) << 10) // :402
#define PTE2PA(pte) (((pte) >> 10) << 12) // :404
#define PTE_FLAGS(pte) ((pte) & 0x3FF) // :406
#define PX(level, va) (((uint64)(va)) >> PXSHIFT(level)) & PXMASK // :411
```

커널 함수 main의 주소 0x80000e60을 손으로 쪼갬 뒤, gdb에서 같은 식으로 커널 페이지 테이블을 따라가 확인했다. 0x80000000은 2 << 30이므로 PX(2)는 2, PX(1)과 PX(0)은 0, 페이지 안 위치는 하위 12비트 0xe60이다.



```
(gdb) set $va = (unsigned long)&main
(gdb) p/x $va
$1 = 0x80000e60
(gdb) p/x ($va >> 30) & 0x1ff
$2 = 0x2
(gdb) p/x ($va >> 21) & 0x1ff
$3 = 0x0
(gdb) p/x ($va >> 12) & 0x1ff
$4 = 0x0
(gdb) p/x $va & 0xfff
$5 = 0xe60
(gdb) set $pte = kernel_pagetable[($va >> 30) & 0x1ff]
(gdb) p/x $pte
$6 = 0x21ff7001
(gdb) p/x $pte & 0x3ff
$7 = 0x1
(gdb) set $pte = ((unsigned long *)($pte >> 10) << 12)[($va >> 21) & 0x1ff]
(gdb) p/x $pte
$8 = 0x21ff6c01
(gdb) set $pte = ((unsigned long *)($pte >> 10) << 12)[($va >> 12) & 0x1ff]
(gdb) p/x $pte
$9 = 0x2000004b
(gdb) p/t $pte & 0x3ff
$10 = 1001011
(gdb) p/x (($pte >> 10) << 12) | ($va & 0xfff)
$11 = 0x80000e60
(gdb)
```

tmux 캡처(gdb). 매크로와 같은 시프트·마스크 식으로 세 단계를 따라갔다.

가상 주소 0x80000e60

63..39	38..30	29..21	20..12	11..0
0	PX(2) = 2	PX(1) = 0	PX(0) = 0	0xe60

$PX(level, va) = (va \gg (12 + 9 * level)) \& 0x1ff$

L0 의 PTE 0x2000004b

63..10 PPN	9..7	6 A · 5 G · 4 U · 3 X · 2 W · 1 R · 0 V
0x80000	0	1 · 0 · 0 · 1 · 0 · 1 · 1

$PTE2PA(pte) = (pte \gg 10) \ll 12 = 0x80000000$, 여기에 0xe60 을 OR 하면 0x80000e60

그림 7. 0x80000e60의 비트 나누기와 마지막 단계 PTE의 해석.

마지막 PTE 0x2000004b의 하위 10비트는 이진수 1001011이다. V(0), R(1), X(3)이 켜져 있고 W는 꺼져 있다. 커널 코드를 읽기·실행만 가능하게 매핑한 `kvmmap(..., PTE_R | PTE_X)(kernel/vm.c:39)`와 맞는다. 6번 비트 A(accessed)도 켜져 있는데, xv6 코드에는 `PTE_A`를 쓰는 곳이 없으므로 하드웨어(QEMU)가 켜 것으로 보인다. 이 부분은 따로 확인하지 않았다. `PTE2PA`로 얻은 0x80000000에 0xe60을 OR하면 원래 주소가 된다. 커널은 물리 주소를 그대로 가상 주소로 쓰기 때문이다(교재 3.2절).

핵심 비트 하나는 플래그 하나다. |로 켜고, &로 확인하고, 시프트와 마스크로 필드를 꺼낸다.

10. 정리

1. .o는 주소가 정해지지 않은 조각이고, 링커가 `user.ld`에 따라 주소를 정한다. 진입점은 `start`다.
2. 사용자 주소 공간은 아래부터 `text`, `rodata`, `data`, `bss`, 가드 페이지, 스택, 힙이다. 앞의 넷은 링커가, 스택과 가드 페이지는 `exec`이, 힙은 `sbrk`가 정한다.
3. 포인터는 주소이고, 타입은 그 주소를 어떻게 읽을지와 +1의 크기를 정한다.
4. 배열 이름은 주소처럼 쓰이지만 `sizeof`는 배열 전체를 쟀다. 문자열은 0으로 끝난다. 배열 밖 접근은 막히지 않는다.
5. `main`의 인자는 `a0`, `a1`로 들어오고, `argv`는 `exec`이 스택에 만든 포인터 배열이다.
6. `kalloc`은 빈 페이지 안에 `next`를 둔 단일 리스트, 버퍼 캐시는 `prev/next`의 이중 리스트다.
7. 플래그와 페이지 테이블 필드는 |, &, 시프트, 마스크로 다룬다.